

CUTTLEFISH_TIER2

Helix OTA — Tier-2 Cuttlefish Virtual-Android OTA Validation

Field	Value
Revision	1
Last modified	2026-06-10T00:00:00Z
Status	designed — host-gated, not yet runnable (no Linux + nested-KVM host attached)
Status summary	The concrete bring-up + integration plan for Tier-2 of the emulated-device testing strategy: a Cuttlefish (cvd) virtual Android device on a Linux + nested-KVM host, used to exercise the real Android A/B update_engine + AVB/dm-verity + auto-rollback flow the Tier-1 podman emulator cannot. Per §11.4.8 (deep-web-research-before-implementation) + §11.4.99 (latest-source cross-reference) — every load-bearing claim is sourced from the latest official Android docs (verified 2026-06-10, see ## Sources verified); uncertain claims are explicitly marked UNCONFIRMED: per §11.4.6 (no-guessing).
Authority	Helix OTA control-plane / device-integration team
Related	docs/design/EMULATED_DEVICE_TESTING.md (the Tier-1/2/3 overview this doc details for Tier-2); containers/ submodule (vasic-digital/containers, §11.4.76); submodules/ota-protocol, submodules/ota-android-agent, submodules/ota-update-engine-bridge; docs/RESUMPTION.md

1. Purpose + scope

Tier-1 (docs/design/EMULATED_DEVICE_TESTING.md) validates the **control-plane protocol** end-to-end: a podman ota-device-emulator speaks the real ota-protocol to a live server and walks

register → update-check → telemetry → delta → rollout → recall.

Tier-1 explicitly does **not** exercise the device's *physical* apply path — there is no real update_engine, no real A/B partitions, no AVB/dm-verity, no auto-rollback.

Tier-2 closes exactly that gap. It stands up a **Cuttlefish virtual Android device** — a high-fidelity, full-Android virtual machine — on a Linux + nested-KVM host, boots a real Android system image built with A/B (and Virtual A/B), and applies a real OTA payload.bin through the real update_engine daemon so that:

- the inactive slot is genuinely written and marked,
- the slot switch genuinely happens on reboot,
- AVB/dm-verity genuinely verifies the new slot, and
- a deliberately-corrupt slot genuinely triggers auto-rollback to the previous slot.

This document is the concrete bring-up + integration recipe. It is the Tier-2 detail behind the Tier-1 doc's §2 “Tier-2 — Cuttlefish virtual Android device” summary.

2. Honest boundary (§11.4.112 / §11.4.6)

Tier-2 is host-gated, NOT structurally impossible. This is a §11.4.112 classification distinction, made as FACT:

- **It runs the moment a Linux + nested-KVM environment is available.** Cuttlefish “can run both remotely (using third-party cloud offerings such as Google Cloud Engine) and locally (on Linux x86 and ARM64 machines)” [SRC-CF-OVERVIEW]. It uses QEMU + KVM hardware-assisted virtualization [SRC-CF-USE]. The blocker is environment provisioning, not a platform/protocol impossibility.
- **The current Apple-Silicon development host cannot run it.** Cuttlefish requires a Debian-based Linux host with KVM available; the documented pre-flight check is `grep -c -w "vmx\|svm" /proc/cpuinfo` (x86, must be nonzero) or `find /dev -name kvm` (ARM64) [SRC-CF-USE]. The Helix dev host is macOS on Apple Silicon using the applehv hypervisor — there is no /dev/kvm, so Cuttlefish does not run there. This is a host limitation, **not** a §11.4.112 structural impossibility.
- **Therefore:** Tier-2 is run on a Linux box / Linux CI runner / a cloud nested-virtualization instance (GCE with nested virt enabled, per [SRC-CF-OVERVIEW] — “Cuttlefish natively supports Google Cloud”). Until such a host is attached, Tier-2 is designed, not yet runnable — the correct §11.4.21 posture is host-availability, not Operator-blocked (the agent can stand it up the moment a Linux+KVM host exists).

Contrast with a true §11.4.112 case: relocating a secure surface to a second display is *structurally impossible*. Tier-2 is the opposite — fully possible, merely host-gated.

3. Host prerequisites (FACT, sourced)

Requirement	Detail	Source
OS	Debian-based Linux (the docs target Debian/Ubuntu host packages)	[SRC-CF-USE]
Virtualization	KVM present and usable. Verify: x86 <code>grep -c -w "vmx\ svm" /proc/cpuinfo</code> returns nonzero; ARM64 <code>find /dev -name kvm</code> finds the node	[SRC-CF-USE]
Cloud variant	Nested virtualization must be enabled on the cloud instance (e.g. GCE nested-virt)	[SRC-CF-OVERVIEW]
Host dev tools	<code>sudo apt install -y git devscripts equivs config-package-dev debhelper-compat golang curl</code> (build deps for the host packages)	[SRC-CF-USE]
Group membership	User in kvm, cvdnetwork, render groups	[SRC-CF-GH]

UNCONFIRMED: the exact apt dependency token is shown as `equivs` in one fetched rendering and `equivs` in the canonical package name; the correct Debian package is **`equivs`**. Verify the literal package list against <https://source.android.com/docs/setup/create/cuttlefish-use> at bring-up time before pasting the apt line.

4. Step-by-step bring-up

All commands run on the Linux + nested-KVM host. Where a literal command is quoted from a source it carries its [SRC-...] tag; commands without a tag are Helix-side orchestration we author.

4.1 Install the Cuttlefish host packages

```
# 1. Get the host-package source
git clone https://github.com/google/android-cuttlefish #
    [SRC-CF-GH]
cd android-cuttlefish

# 2. Build the Debian host packages
```

```

tools/buildutils/build_packages.sh                                     #
    [SRC-CF-GH]

# 3. Install base + user packages
sudo apt install ./cuttlefish-base_*.deb ./cuttlefish-
    user_*.deb    # [SRC-CF-GH]

# 4. Join the required groups, then reboot to load kernel modules
    + udev rules
sudo usermod -aG kvm,cvdnetwork,render "$USER"                       #
    [SRC-CF-GH]
sudo reboot                                                         #
    [SRC-CF-USE] (reboot loads modules + udev rules)

```

Notes (FACT): cuttlefish-base is **required**, cuttlefish-user is **recommended**; cuttlefish-common is **deprecated** (compatibility shim depending on base+user) [SRC-CF-GH]. The reboot is mandatory — it triggers installing additional kernel modules and applying udev rules [SRC-CF-USE]. An Artifact-Registry apt path also exists as an alternative to building from source [SRC-CF-GH].

4.2 Obtain a device image + host package with A/B

Cuttlefish device builds are published on the Android CI site [SRC-CF-USE]:

1. Go to <http://ci.android.com/> and select branch **aosp-android-latest-release** (per the 2026 trunk-stable model, use android-latest-release rather than aosp-main [SRC-CF-USE]).
2. Choose build target **aosp_cf_x86_64_only_phone** (x86_64 host) or **aosp_cf_arm64_only_phone** (ARM64 host) [SRC-CF-USE].
3. Download the **userdebug** artifacts:
 - Device images: aosp_cf_x86_64_phone-img-xxxxxx.zip (or the ARM64 variant) [SRC-CF-USE]
 - Host package: cvd-host_package.tar.gz from the **same build** [SRC-CF-USE]

UNCONFIRMED: whether the default aosp_cf_* targets ship **Virtual A/B** vs legacy A/B by default for the chosen release. Virtual A/B writes new data to a COW device and merges dynamic partitions post-reboot via dm-user + snapuserd [SRC-VAB]; legacy A/B writes the inactive physical slot directly [SRC-AB]. The OTA-apply mechanics in §5 are identical at the update_engine level; only the underlying partition mechanism differs. Verify which is active on the chosen image via getprop ro.virtual_ab.enabled on the booted device before authoring the rollback assertions.

4.3 Launch the virtual device

```

# In a clean directory, extract both archives, then:
HOME=$PWD ./bin/launch_cvd --daemon                                # [SRC-CF-USE]
./bin/adb devices                                                    # [SRC-CF-USE] -
    device should appear
# Web UI (operator inspection): https://localhost:8443             [SRC-CF-
    USE]

```

UNCONFIRMED: the current CLI is migrating from `launch_cvd/stop_cvd` to a unified **cvd** front-end (`cvd start / cvd stop`). The fetched cuttlefish-use page documents `launch_cvd --daemon + stop_cvd` as the canonical commands [SRC-CF-USE]; the AOSP cvd front-end was not confirmed in the pages fetched. Use `launch_cvd/stop_cvd` as documented; if the host packages expose `cvd`, prefer it and update this doc with the verified command. Do not guess the `cvd` subcommand surface.

4.4 Stop the virtual device

```
HOME=$PWD ./bin/stop_cvd # [SRC-CF-USE]
```

5. Applying + observing a real OTA on Cuttlefish

This is the core of Tier-2 — the path Tier-1 cannot exercise.

5.1 How update_engine applies the payload (FACT)

- `update_engine` writes to the **inactive slot** while the system runs from the active slot; it processes the OTA package as an ordered sequence of metadata operations, downloading each operation's data to memory, applying it, and discarding the memory [SRC-AB]. `payload.bin` (the bulk of an OTA zip) is **streamed** directly to the inactive slot without temporary `/data` storage [SRC-AB].
- Slot state is managed by the **boot_control HAL** via three attributes — *bootable*, *active/preferred*, *successful*. During apply, the inactive slot is marked unbootable (`setSlotAsUnbootable()`), then marked active (`setActiveBootSlot()`) after the payload applies successfully [SRC-AB].
- After writing, the partition is **re-read, hashed, and compared** to the expected hash from the metadata; `update_engine` checkpoints the last operation so it can resume after an interruption [SRC-AB] [SRC-UE-README].

5.2 Driving the apply (FACT)

Two documented, Cuttlefish-supported paths [SRC-UE-README] [SRC-UE-SEARCH]:

```
# Path A – serve a full OTA zip to the adb-connected Cuttlefish
#           device over HTTP
python system/update_engine/scripts/update_device.py <path-to-ota.zip> # [SRC-UE-README]
# ("Use the scripts/update_device.py program and pass a path to
#  your OTA zip file";
#  it serves payload.bin over HTTP to the device via ADB – no
#  full server needed.)

# Path B – trigger update_engine directly on the device (when a
#           URL is already wired)
adb shell update_engine_client --interactive=false --
    check_for_update # [SRC-UE-README]
```

FACT [SRC-UE-SEARCH]: “Cuttlefish works as well for testing `update_engine`”; the `update_engine_unittests` require an adb-connected device and run on Cuttlefish.

5.3 Observing the slot switch + AVB/dm-verity + rollback (FACT + assertions)

- **Slot switch on reboot.** After apply, the device boots into the updated partition on next reboot; if `ab_config.force_switch_slot` is true a “switch slot” control is enabled, otherwise the next boot lands on the updated slot (or the user manually sets the active slot) [SRC-UE-SEARCH]. **Assert:** `adb shell getprop ro.boot.slot_suffix` flips (e.g. `_a` → `_b`) across the reboot; `adb shell bootctl get-current-slot` reflects the new slot. > **UNCONFIRMED:** exact availability of `bootctl` as a shell command on the chosen > Cuttlefish image vs reading slot state only via `getprop ro.boot.slot_suffix / > dumphsys`. Verify on the booted device; do not assert a command that may be absent.
- **AVB / dm-verity verification.** dm-verity “guarantees a device will boot an uncorrupted image”; on a bad OTA or dm-verity failure the device “can reboot into an old image” [SRC-AB-SEARCH]. The `update_verifier` daemon performs post-reboot integrity checks using dm-verity **before** the slot is marked successful [SRC-AB]. **Assert:** a clean apply marks the new slot *successful* (boot completes, no rollback); observe `update_verifier` in `logcat`.
- **Auto-rollback on a corrupt slot.** A/B keeps the unused slot as a fallback: “If an error occurs during or immediately after an update, the system can rollback to the old slot and continue to have a working system” [SRC-AB-SEARCH]. If a non-successful slot fails to boot repeatedly, the bootloader marks it unbootable and switches back [SRC-AB]. **Assert (the headline Tier-2 proof):** deliberately corrupt the just-written inactive slot (or apply a payload that fails verification), reboot, and capture that the device **rolls back** to the previous good slot (`ro.boot.slot_suffix` returns to the original), with `update_verifier` / dm-verity evidence in `logcat`. > **UNCONFIRMED:** the exact, safe mechanism to corrupt the inactive slot on a Virtual > A/B Cuttlefish image (COW device + `snapperd`) without harming the host. On legacy > A/B this is a direct write to the inactive partition image; on Virtual A/B it must > target the COW/merge path. Establish the precise method from [SRC-VAB] + > [SRC-VAB-PATCHES] at bring-up and record it as FACT before running — never guess a > destructive write (composes §11.4.133 target-hardware-safety: the “target” here is > the virtual device, but the discipline of verified-before-destructive-write holds).

6. Helix integration — control plane + device agent (no new protocol)

Tier-2’s value is that the **device side speaks the same ota-protocol the Tier-1 emulator does**, but now backed by a **real `update_engine` target** instead of a stub.

- **Control plane (server/).** Unchanged. The Cuttlefish device registers and polls `/api/v1` exactly like the Tier-1 emulator and a real RK3588 board — register → update-check → telemetry → delta → rollout → recall. The server cannot tell a Cuttlefish client from a real board at the protocol layer; that is the point — same wire contract, higher device-side fidelity.

- **Device agent (submodules/ota-android-agent + submodules/ota-update-engine-bridge).** On Cuttlefish these run against the **real** update_engine daemon:
 - ota-android-agent makes the same DeltaApplyDecision it makes everywhere, then
 - ota-update-engine-bridge hands the resolved payload to the **real** update_engine (via update_engine_client / update_device.py per §5.2) instead of a no-op.
 - The agent observes the **real** slot switch + update_verifier result and reports real apply/rollback telemetry back up the ota-protocol to the server.
- **What this proves that Tier-1 cannot:** the ota-update-engine-bridge → real update_engine → real slot/AVB/rollback path. The Tier-1 emulator stubs this entire segment; Tier-2 is the first tier to exercise it without physical hardware.

6.1 Evidence flow (anti-bluff, §11.4.69 / §11.4.107)

Every Tier-2 run captures under docs/qa/<run-id>/:

- the full ota-protocol request/response transcript (as Tier-1 does), **plus**
- on-device apply evidence: update_engine logcat, pre/post ro.boot.slot_suffix, update_verifier outcome, and the rollback capture for the corrupt-slot case.

A Tier-2 PASS without the **on-device slot-switch + rollback** evidence is a §11.4 PASS-bluff — it would be claiming the device-apply coverage that is Tier-2’s entire reason to exist. The slot-suffix delta + update_verifier logcat are the §11.4.108 runtime signatures for “the OTA really applied / really rolled back.”

7. Wiring into the containers submodule + CI (§11.4.76 / §11.4.74)

Per §11.4.76 (Containers-submodule mandate) and §11.4.74 (extend-don’t-reimplement), the Cuttlefish lifecycle is orchestrated **through the vasic-digital/containers submodule**, not reimplemented in helix_ota.

- **Existing seams the submodule already provides** (FACT, from containers/pkg/):
 - pkg/boot + pkg/compose + pkg/health — on-demand boot, compose orchestration, readiness gating (the same path Tier-1 uses; satisfies the §11.4.76 on-demand-infra invariant — the test entry point boots the infra, operators never hand-start anything).
 - pkg/emulator — Android emulator orchestration **with KVM-acceleration gating** (accel.go), containerized-KVM support (containerized_kvm_test.go), AVD-lock clearing, and orphan qemu-system-* reaping. This is the closest existing primitive to a Cuttlefish lifecycle and the natural place to extend.
 - pkg/vm — QEMU VM orchestration (aarch64 -machine virt + AAVMF UEFI, qemu.go), the seam for VM-shaped targets on a KVM host.
- **Extension (PR upstream, never in-project) — proposed pkg/cuttlefish:** a thin cvd lifecycle wrapper exposing the submodule’s standard lifecycle surface

(boot / health / teardown) over `launch_cvd --daemon + stop_cvd + adb devices` readiness. It reuses `pkg/emulator`'s KVM-accel gating and orphan-reaping, and `pkg/health` for the “adb devices shows the device” readiness probe. The **OTA-domain logic stays in `helix_ota`** (the ota-device agent speaks ota-protocol); only the Cuttlefish boot/health/teardown plumbing is the submodule's job. Record the survey result on the tracker row as Catalogue-Check: extend vasic-digital/containers@<sha> per §11.4.74.

- **CI wiring (FACT-gated):** the Tier-2 job runs **only** on a Linux + nested-KVM runner. The job: (1) §3 pre-flight (`grep vmx/svm` or `/dev/kvm` present, else SKIP-with-reason per §11.4.3 — never PASS-by-default), (2) boot Cuttlefish via the submodule's `pkg/cuttlefish` seam, (3) bring up the control plane (the same compose stack Tier-1 uses), (4) run the §5 apply + observe assertions on both a clean-apply and a corrupt-slot scenario, (5) capture evidence to `docs/qa/<run-id>/`,
 1. teardown via `stop_cvd`. On a non-KVM runner (the macOS dev host included), the job SKIPS with the host-gated reason — it is **not** a failure and **not** silently green.

8. Open items to verify at bring-up (no-guessing register, §11.4.6)

Each UNCONFIRMED: above is a verify-before-run item. Consolidated:

1. Exact apt build-dep list (equivs spelling) — verify against [SRC-CF-USE].
2. Whether the chosen `aosp_cf_*` release defaults to Virtual A/B vs legacy A/B — `getprop ro.virtual_ab.enabled` on the booted device.
3. Current CLI surface — `launch_cvd/stop_cvd` (documented) vs the unified cvd front-end (unconfirmed in fetched pages).
4. Slot-state read command on the chosen image — `bootctl` availability vs `getprop ro.boot.slot_suffix` only.
5. The safe, exact corrupt-the-inactive-slot mechanism for the chosen A/B variant (legacy direct-write vs Virtual A/B COW path) — from [SRC-VAB] / [SRC-VAB-PATCHES].

None of these block the *design*; all five are FACT-establishment steps the runner must complete (and capture as evidence) before the first Tier-2 PASS is claimed.

Sources verified 2026-06-10

- [SRC-CF-USE] Get started — Cuttlefish, Android Open Source Project — <https://source.android.com/docs/setup/create/cuttlefish-use> (host OS/KVM checks, build-dep apt line, reboot, CI image targets, `aosp-android-latest-release`, `launch_cvd --daemon`, `adb devices`, <https://localhost:8443>, `stop_cvd`) — accessed 2026-06-10.
- [SRC-CF-OVERVIEW] Cuttlefish virtual Android devices, AOSP — <https://source.android.com/docs/devices/cuttlefish> (runs locally on Linux x86/ARM64 and remotely on GCE; native Google Cloud support; full-fidelity with the Android framework) — accessed 2026-06-10.
- [SRC-CF-GH] google/android-cuttlefish host packages README — <https://github.com/google/android-cuttlefish/tree/main/tools/buildutils/>

- `build_packages.sh, apt install ./cuttlefish-base_*.deb ./cuttlefish-user_*.deb, usermod -aG kvm, cvdnetwork, render, cuttlefish-common deprecated, Artifact-Registry path)` — accessed 2026-06-10.
- [SRC-AB] A/B (seamless) system updates, AOSP — <https://source.android.com/docs/core/ota/ab> (update_engine writes inactive slot, streams payload.bin, boot_control HAL bootable/active/successful, setSlotAsUnbootable/setActiveBootSlot, post-write re-read+hash, checkpointing, update_verifier dm-verity post-reboot check) — accessed 2026-06-10.
 - [SRC-AB-SEARCH] A/B updates + rollback summary (AOSP A/B + dm-verity docs surfaced via search) — A/B keeps the unused slot as fallback; rollback to old slot on error; dm-verity guarantees uncorrupted boot / reboot into old image on bad OTA — accessed 2026-06-10.
 - [SRC-UE-README] platform/system/update_engine README, Git at Google — https://android.googlesource.com/platform/system/update_engine/+/-/master/README.md (update_engine_client --interactive=false --check_for_update; scripts/update_device.py <ota.zip> serves payload over HTTP via adb; checkpointing) — accessed 2026-06-10.
 - [SRC-UE-SEARCH] update_engine + Cuttlefish testing notes (Git at Google update_engine pages + updater_sample README surfaced via search) — “Cuttlefish works as well for testing update_engine”; update_engine_unittests need an adb device; ab_config.force_switch_slot slot-switch behaviour — accessed 2026-06-10.
 - [SRC-VAB] Virtual A/B overview, AOSP — https://source.android.com/docs/core/ota/virtual_ab (OTA writes to a COW device; dynamic partitions merged post-reboot via dm-user + snapuserd) — accessed 2026-06-10.
 - [SRC-VAB-PATCHES] Implement Virtual A/B — patches, AOSP — https://source.android.com/docs/core/ota/virtual_ab/implement-patches (Virtual A/B implementation detail referenced for the corrupt-slot mechanism verification) — accessed 2026-06-10.

Negative findings / gaps (§11.4.99(B))

- The fetched cuttlefish overview page did **not** explicitly restate the KVM/nested-virt requirement in its excerpt (it states Linux x86/ARM64 + GCE); the KVM requirement is confirmed by [SRC-CF-USE]’s pre-flight checks. Absence on one page is not contradiction.
- The unified cvd CLI (`cvd start/cvd stop`) was **not** confirmed in the fetched pages; `launch_cvd/stop_cvd` is the documented surface used here. Tracked as open item §8.3 — do not assume cvd subcommands.